

PROGRAM- 9

Implementation of Gaussian Mixture Model (GMM) using Expectation–Maximization (EM)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generate sample dataset
X, y = make_blobs(n_samples=300,
                  centers=3,
                  cluster_std=1.5,
                  random_state=42)

# Number of clusters
k = 3

# Number of features
n = X.shape[1]

# Initialize parameters
np.random.seed(42)

means = X[np.random.choice(len(X), k, False)]
covariances = [np.eye(n) for _ in range(k)]
weights = np.ones(k) / k

# Gaussian function
def gaussian(x, mean, cov):
    det = np.linalg.det(cov)
    inv = np.linalg.inv(cov)

    norm_const = 1.0 / (
        np.power((2 * np.pi), n / 2) *
        np.sqrt(det)
    )

    diff = x - mean

    result = np.exp(
        -0.5 * np.sum(
            diff @ inv * diff,
            axis=1
        )
    )
```

```
)
)

return norm_const * result
```

```
# EM Algorithm
iterations = 10
```

```
for iteration in range(iterations):
```

```
    # E-step
    responsibilities = np.zeros((len(X), k))
```

```
    for i in range(k):
        responsibilities[:, i] = (
            weights[i] *
            gaussian(X,
                    means[i],
                    covariances[i])
        )
```

```
    responsibilities = (
        responsibilities /
        responsibilities.sum(axis=1,
                            keepdims=True)
    )
```

```
    # M-step
    Nk = responsibilities.sum(axis=0)
```

```
    for i in range(k):
```

```
        means[i] = (
            responsibilities[:, i]
            @ X
        ) / Nk[i]
```

```
        diff = X - means[i]
```

```
        covariances[i] = (
            (responsibilities[:, i]
             * diff.T)
            @ diff
        ) / Nk[i]
```

```
        weights[i] = Nk[i] / len(X)
```

```
# Assign clusters
labels = np.argmax(responsibilities,
                  axis=1)

# Plot result
plt.scatter(X[:, 0],
           X[:, 1],
           c=labels)

plt.title(
    "Gaussian Mixture Model using EM"
)

plt.show()
```

OUTPUT-

